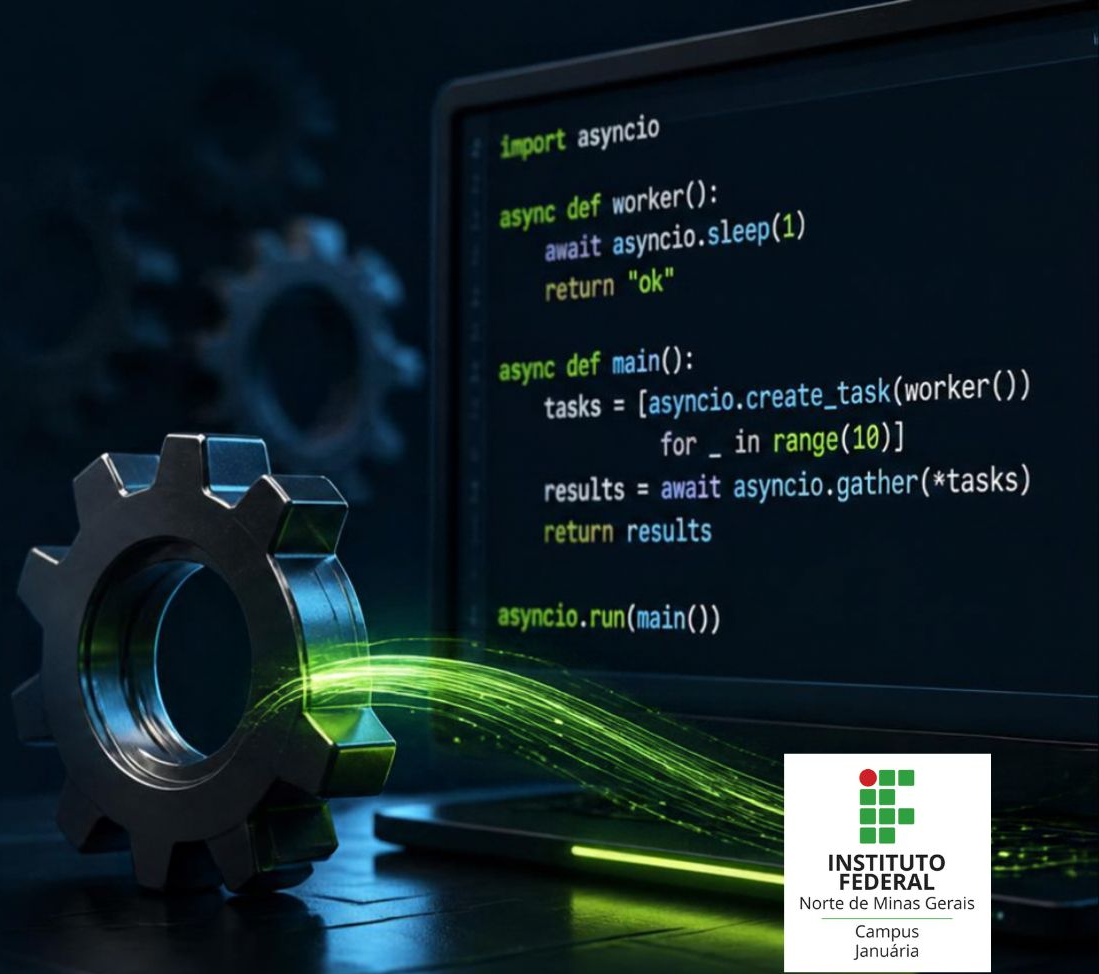
Funções Geradoras e Corrotinas Assíncronas (asyncio)



Gabriel Fernandes dos Santos

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

O que são Geradores?

Funções especiais que preservam o estado da execução.

Usam yield para pausar e retomar a execução.

Ceder em vez de finalizar: continuam de onde pararam.




```
def geradora(ds):
 for item in ds:
   yield item
```

Vantagens e Quando Usar Geradores

OTIMIZAÇÃO DE MEMÓRIA

Evita carregar grandes volumes na RAM, gera e cede um item por vez

USO INDICADO

Grandes volumes de dados
Sequências infinitas
Processamento linha a linha

TRADICIONAL	GERADORA

`def potencia(n):`
`  result = []`
`  for i in range(n):`
`  result.append(2**i)`
`  return result`
	`def gerador_potencia(n):`
	`  for i in range(n):`
	`  yield 2**i`

RAM  CHEIA	RAM  VAZIA

Geradores x Iteradores

Característica	Iterador	Gerador
Definição	Objeto que implementa __iter__ e __next__.	Função que usa yield para produzir uma sequência.
Memória	Armazena todos os valores na memória.	Gera valores sob demanda (lazy evaluation).
Criação	Criado a partir de uma classe ou coleção existente.	Criado a partir de uma função geradora.
Estado	Mantém o estado da iteração atual.	Mantém o estado da execução da função (variáveis locais).
Relação	Pode ser um gerador (se implementado assim).	Sempre é um iterador.

O que são Corrotinas Assíncronas?

Concorrência leve baseada em funções geradoras.

Definidas com async def e pausadas com await.

Executam apenas quando agendadas no Event Loop.



Event Loop

Laço infinito que gerencia a execução assíncrona.
Executa uma tarefa até encontrar await.
Não depende do SO: é gerenciado pela aplicação, em single-thread.



Async IO

asyncio é uma biblioteca da biblioteca padrão do Python voltada à programação assíncrona. Ela fornece recursos para executar e gerenciar corrotinas de forma concorrente por meio de um Event Loop

- async def — define corrotinas assíncronas.
- await — suspende a execução da corrotina enquanto uma operação é aguardada.
- asyncio.run() — inicia e executa a corrotina principal.
- asyncio.create_task() — agenda corrotinas para execução concorrente.

Sync IO

Gerenciamento de Tarefas
Principais métodos para controle de corrotinas.



asyncio.gather()

executa várias corrotinas simultaneamente e retorna resultados quando todas terminam



asyncio.TaskGroup()

agrupa tarefas com cancelamento seguro em caso de falha



asyncio.wait()

espera conclusão de conjunto de tarefas (todas ou a primeira)



asyncio.wait_for()

define timeout para execução de uma tarefa

Vantagens e Limitações

VANTAGENS

Eficiência para operações I/O-bound (rede, disco)

Escalabilidade: milhares de corrotinas rodando em uma única thread

LIMITAÇÕES

Não é paralelismo real (não usa múltiplos núcleos)

Bibliotecas **bloqueantes travam** o Event Loop

Muitas Tasks podem **aumentar** a complexidade e o custo de gerenciamento.